

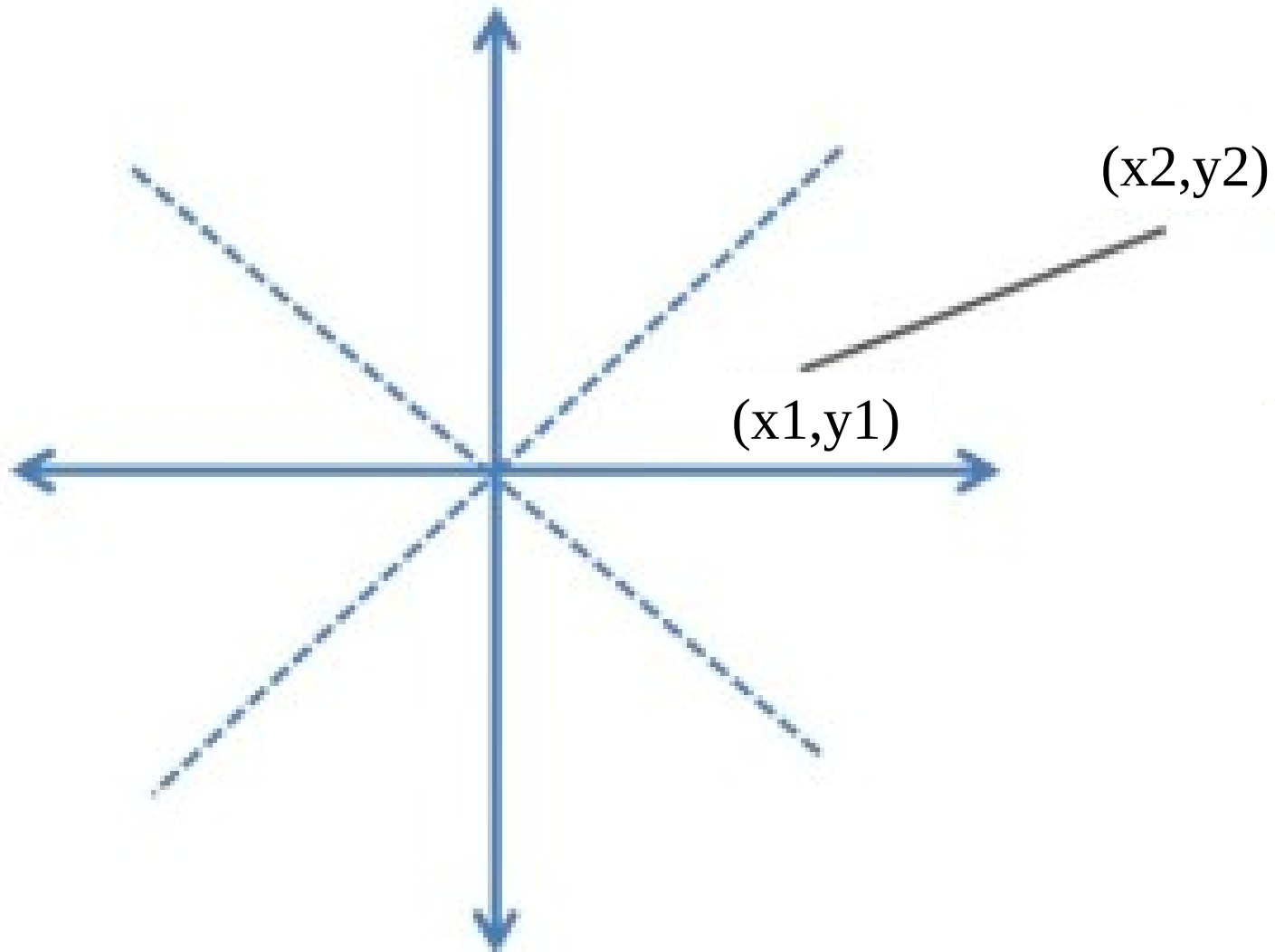
Scan line Conversion

Slope, DDA, Bresenham, Midpoint
Circle, Ellipse

Line Rendering

- Given two end points of a line, we must be able to draw line.
- To draw a line, we need to find out all the points between two end points.
- These points are equivalent to the pixels on the screen.
- Then, the points are stricked with the electrons so than line is visible on the screen.
- This is done repeatedly.

Line Rendering



Line Rendering

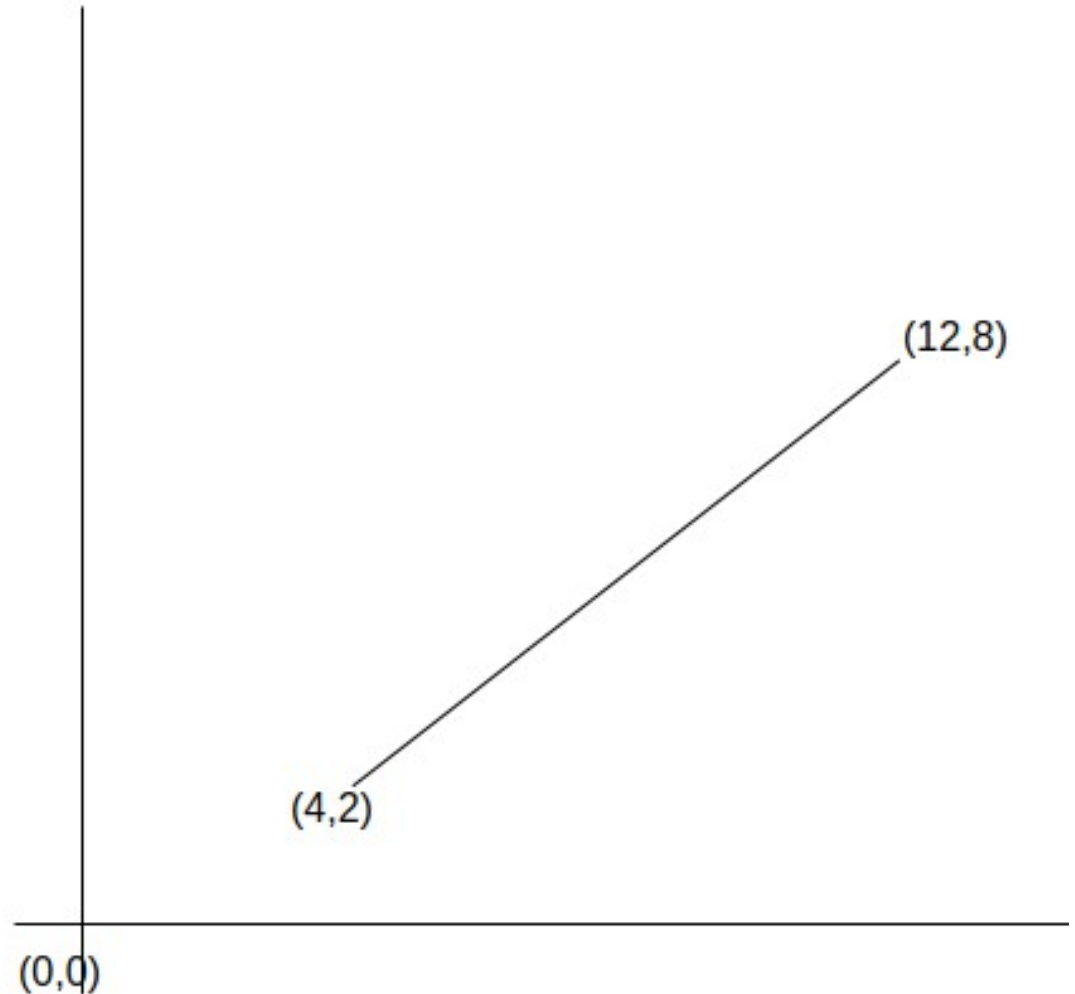
- Let us consider two end points (x_1, y_1) and (x_2, y_2) of the line to be rendered.
- To render this line, we need to compute its equation, $y = mx + c$
- Where, $m = dy/dx$, $dy = y_2 - y_1$ and $dx = x_2 - x_1$
- $c = y_1 - mx_1$
- Now, to find points on the line, we can increment or decrement x-coordinate and workout for y-coordinate.
- We will increment x-coordinate if $x_1 < x_2$ otherwise decrement x-coordinate.

Line Rendering

- What if we increment or decrement y-coordinate and workout for x-coordinate.
- This is also possible.
- But decision must be taken on based on its slope.
- If $|dx| > |dy|$ i.e. $|m| < 1$, then increment or decrement x-coordinate.
- Otherwise increment or decrement y-coordinate.
- This helps to calculate sufficient number of points on the line.
- And line will be smooth

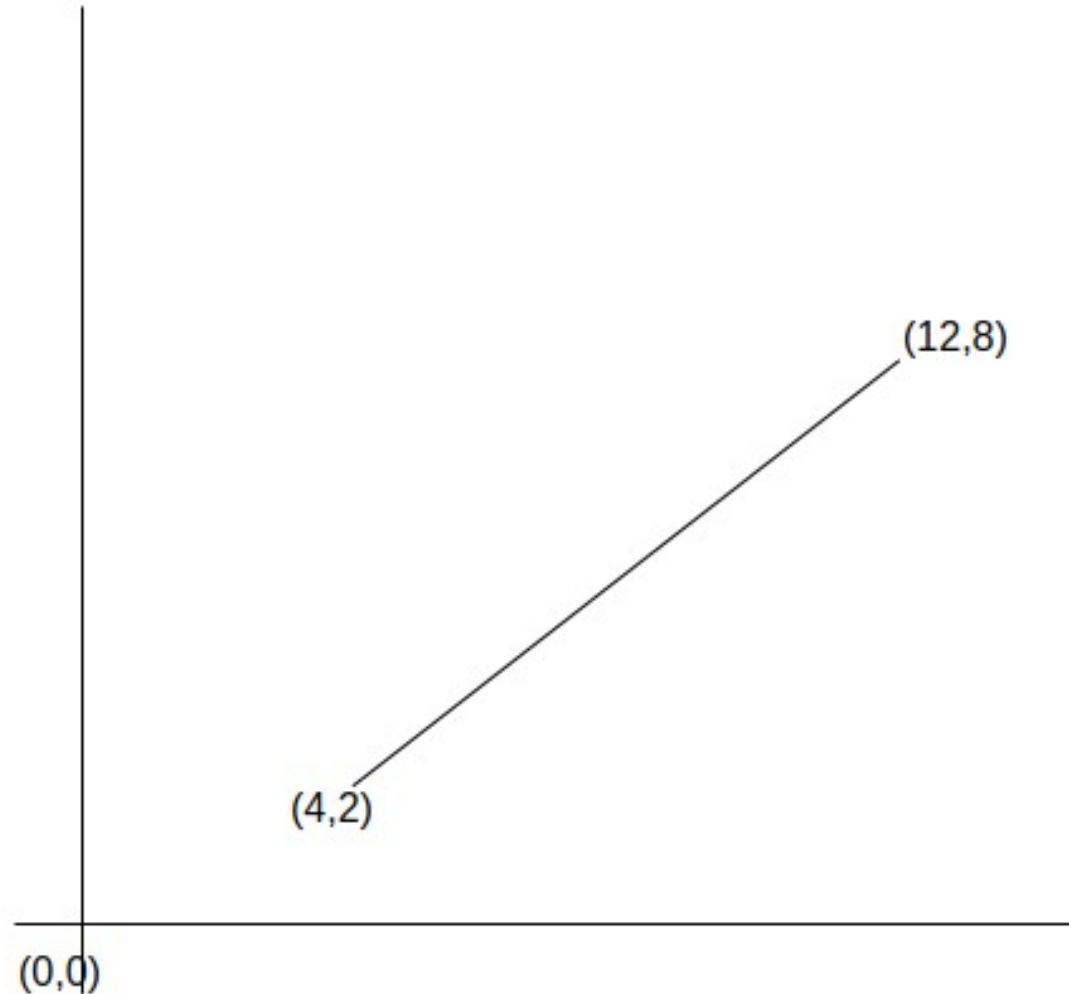
Line Rendering

- $(x_1, y_1) = (4, 2)$
- $(x_2, y_2) = (12, 8)$
- $|dx| = |12 - 4| = 8$
- $|dy| = |8 - 2| = 6$
- Since, $|dx| > |dy|$,
and $x_1 < x_2$,
we increment x



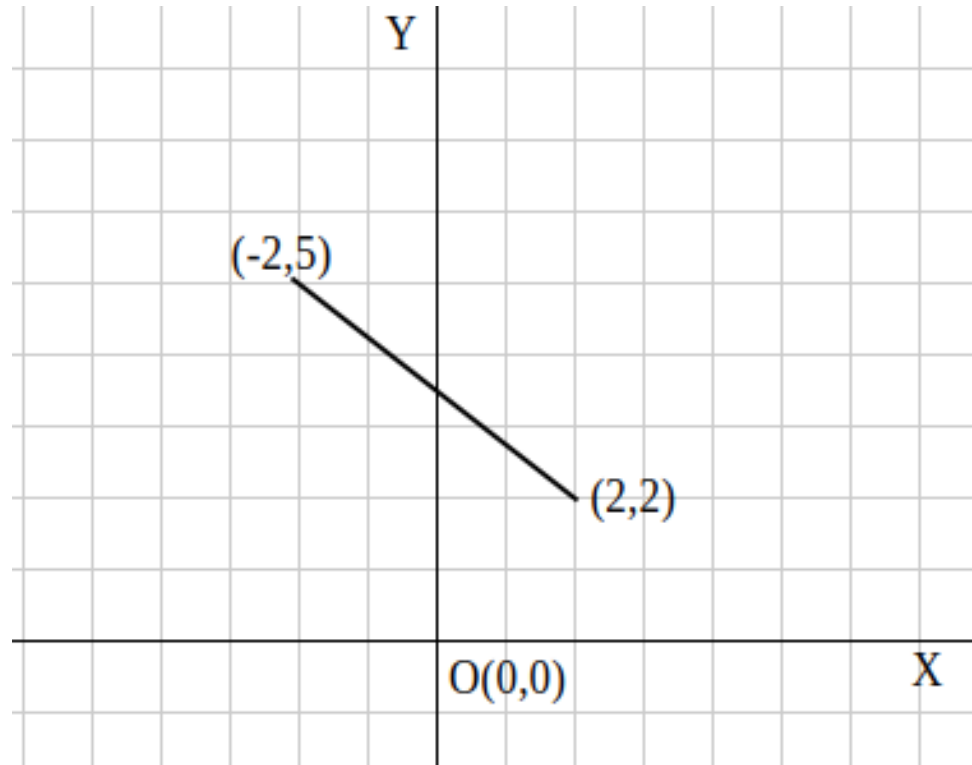
Line Rendering

- $(x_1, y_1) = (12, 8)$
- $(x_2, y_2) = (4, 2)$
- $|dx| = |4 - 12| = 8$
- $|dy| = |2 - 8| = 6$
- Since, $|dx| > |dy|$,
and $x_1 > x_2$,
we decrement x



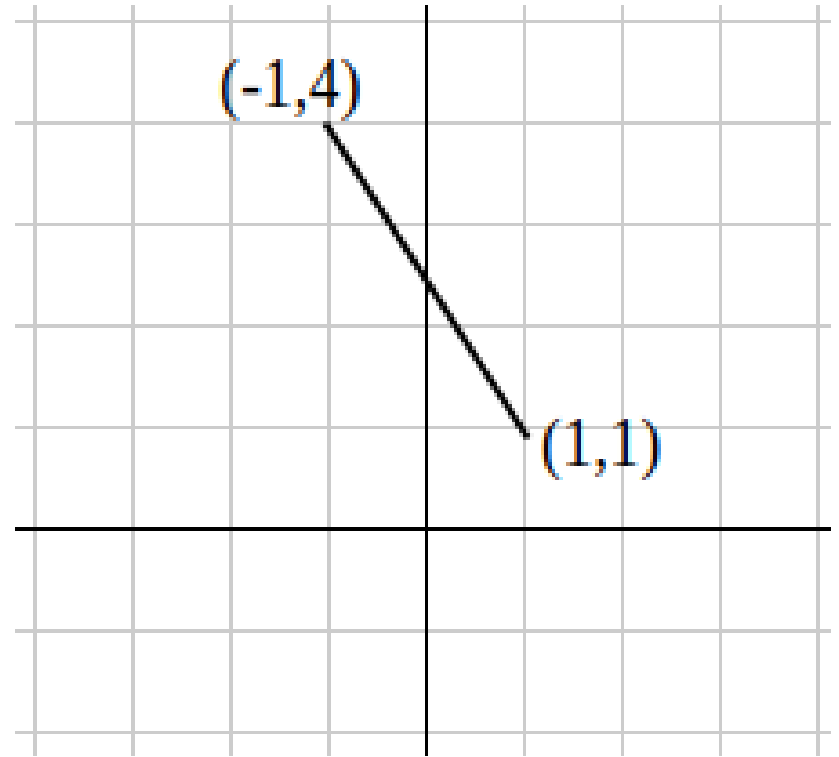
Line Rendering

- $(x_1, y_1) = (-2, 5)$
- $(x_2, y_2) = (2, 2)$
- $|dx| = |2 - (-2)| = 4$
- $|dy| = |2 - 5| = 3$
- Since, $|dx| > |dy|$,
and $x_1 < x_2$
we increment x



Line Rendering

- $(x_1, y_1) = (-1, 4)$
- $(x_2, y_2) = (1, 1)$
- $|dx| = |1 - (-1)| = 2$
- $|dy| = |1 - 4| = 3$
- Since, $|dy| > |dx|$,
and $y_1 > y_2$
we decrement y



Line Rendering Methods

- Mainly there are three methods.
- Direct Method
- DDA
- Bresenham

Direct method

- `directMethod(x1,y1,x2,y2)`
- 1. $dx = x2 - x1$, $dy = y2 - y1$
- 2. $m = dy/dx$, $c = y1 - mx1$
- 3. IF $|dx| > |dy|$, $step = |dx|$ and $inc = dx/step$
- 4. ELSE $step = |dy|$ and $inc = dy/step$
- 5. $x = x1$, $y = y1$ and $i = 0$

Direct method

- 6. `drawPixel(round(x),round(y))`
- 7. IF $|dx| > |dy|$
- 8. WHILE $i < \text{step}$
- 9. $x = x + \text{inc}$, $y = mx + c$
- 10. `drawPixel(round(x),round(y))`
- 11. $i += 1$
- 12. END WHILE
- 13. ELSE

Direct method

- 14. WHILE $i < \text{step}$
- 15. $y = y + \text{inc}$, $x = (y - c) / m$
- 16. `drawPixel(round(x), round(y))`
- 17. $i += 1$
- 18. END WHILE
- 19. END IF

Direct method

- $(x_1, y_1) = (2, 3)$ and $(x_2, y_2) = (7, 5)$
- $dx = x_2 - x_1 = 7 - 2 = 5$
- $dy = y_2 - y_1 = 5 - 3 = 2$
- $m = dy/dx = 2/5 = 0.4$
- $c = y_1 - m \cdot x_1 = 3 - 0.4 \cdot 2 = 2.2$
- Since, $|dx| > |dy|$,
- $step = |dx| = 5$
- $inc = dx/step = 1$
- $x = x_1, y = y_1, i = 0$

i	x+=inc	y=mx+c	round(y)	Output(x,y)
	2	3.00	3	(2,3)
0	3	3.40	3	(3,3)
1	4	3.80	4	(4,4)
2	5	4.20	4	(5,4)
3	6	4.60	5	(6,5)
4	7	5.00	5	(7,5)

Short-comings of Direct Method

- Float multiplication, division, addition, subtraction and round operation
- Need to workout for either x or y in each step

DDA

- Stands for Digital Differential Analyzer.
- An increment approach to speed up line scan conversion.
- It increments or decrement either x or y by 1 based upon dx and dy .
- Then it increment or decrement another co-ordinate by slope or reciprocal of slope.
- It means it avoids explicit calculation of another variable.
- The increment or decrement is determined by the direction of change in x or y

DDA

- $\text{DDA}(x_1, y_1, x_2, y_2)$
- 1. $dx = x_2 - x_1$, $dy = y_2 - y_1$
- 2. IF $|dx| > |dy|$, $\text{step} = |dx|$
- 3. ELSE $\text{step} = |dy|$
- 4. $x_{\text{inc}} = dx / \text{step}$, $y_{\text{inc}} = dy / \text{step}$
- 5. $x = x_1$, $y = y_1$ and $i = 0$

DDA

- 6. drawPixel(round(x),round(y))
- 7. WHILE i<step
- 8. $x=x+xinc$, $y=y+yinc$
- 9. drawPixel(round(x),round(y))
- 10. $i+=1$
- 11. END WHILE

DDA

- $(x_1, y_1) = (2, 3)$ and $(x_2, y_2) = (7, 5)$
- $dx = x_2 - x_1 = 7 - 2 = 5$
- $dy = y_2 - y_1 = 5 - 3 = 2$
- Since, $|dx| > |dy|$,
- $step = |dx| = 5$
- $xinc = dx / step = 5 / 5 = 1$
- $yinc = dy / step = 2 / 5 = 0.4$
- $x = x_1, y = y_1, i = 0$

i	$x += xinc$	$y += yinc$	round(y)	(x, y)
	2	3.00	3	(2, 3)
0	$2 + 1 = 3$	$3 + 0.4 = 3.40$	3	(3, 3)
1	$3 + 1 = 4$	3.80	4	(4, 4)
2	$4 + 1 = 5$	4.20	4	(5, 4)
3	$5 + 1 = 6$	4.60	5	(6, 5)
4	$6 + 1 = 7$	5.00	5	(7, 5)

DDA

- $(x_1, y_1) = (7, 5)$ and $(x_2, y_2) = (2, 3)$
- $dx = x_2 - x_1 = 2 - 7 = -5$
- $dy = y_2 - y_1 = 3 - 5 = -2$
- Since, $|dx| > |dy|$,
- $step = |dx| = 5$
- $xinc = dx / step = -5 / 5 = -1$
- $yinc = dy / step = -2 / 5 = -0.4$
- $x = x_1, y = y_1, i = 0$

i	$x += xinc$	$y += yinc$	round(y)	(x,y)
	7	5.00	5	(7,5)
0	6	4.60	5	(6,5)
1	5	4.20	4	(5,4)
2	4	3.80	4	(4,4)
3	3	3.40	3	(3,3)
4	2	3.00	3	(2,3)

DDA

- $(x_1, y_1) = (2, 3)$ and $(x_2, y_2) = (1, 9)$
- $dx = x_2 - x_1 = 1 - 2 = -1$
- $dy = y_2 - y_1 = 9 - 3 = 6$
- Since, $|dy| > |dx|$,
- $step = |dy| = 6$
- $xinc = dx / step = -1/6 = -0.167 = -0.17$
- $yinc = dy / step = 6/6 = 1$
- $x = x_1, y = y_1, i = 0$

i	$x += xinc$	$y += yinc$	round(x)	(x,y)
	2.00	3	2	(2,3)
0	$2 - 0.17 = 1.83$	$3 + 1 = 4$	2	(2,4)
1	$1.83 - 0.17 = 1.66$	$4 + 1 = 5$	2	(2,5)
2	$1.66 - 0.17 = 1.49$	$5 + 1 = 6$	1	(1,6)
3	$1.49 - 0.17 = 1.32$	$6 + 1 = 7$	1	(1,7)
4	$1.32 - 0.17 = 1.15$	$7 + 1 = 8$	1	(1,8)
5	$1.15 - 0.17 = 0.98$	$8 + 1 = 9$	1	(1,9)

DDA

- $(x_1, y_1) = (-1, 4)$ and $(x_2, y_2) = (1, -2)$
- $dx = x_2 - x_1 = 1 - (-1) = 2$
- $dy = y_2 - y_1 = -2 - 4 = -6$
- Since, $|dy| > |dx|$,
- $step = |dy| = 6$
- $xinc = dx / step = 2 / 6 = 0.33$
- $yinc = dy / step = -6 / 6 = -1$
- $x = x_1, y = y_1, i = 0$

i	$x += xinc$	$y += yinc$	round(x)	(x,y)
	-1.00	4	-1	(-1,4)
0	-0.67	3	-1	(-1,3)
1	-0.34	2	0	(0,2)
2	-0.01	1	0	(0,1)
3	0.32	0	0	(0,0)
4	0.65	-1	1	(1,-1)
5	0.98	-2	1	(1,-2)

Pros and Cons of DDA

- Eliminates floating point multiplication ($m \cdot x$) and division $(y - c)/m$
- But, it still has to perform float addition, subtraction and round operations
- Line is not smooth

Bresenham Algorithm

- Bresenham also increments or decrements one coordinate based upon its slope.
- Another coordinate is sampled from two possible points based upon their closeness to the line.
- It means it uses only integers to scan a line.
- The sampled coordinate might be same or increased or decreased.
- The closeness of two possible points is determined using a decision parameter.

Bresenham Algorithm

We have, initial point (x_0, y_0) and final point (x_n, y_n) .

$$dx = |x_n - x_0|$$

$$dy = |y_n - y_0|$$

Let us suppose that $dx > dy$ i.e. $|m| < 1$

Let $x_n > x_0$ and $y_n > y_0$.

Also suppose that we have computed and plotted all the points from point (x_0, y_0) through some point (x_k, y_k) .

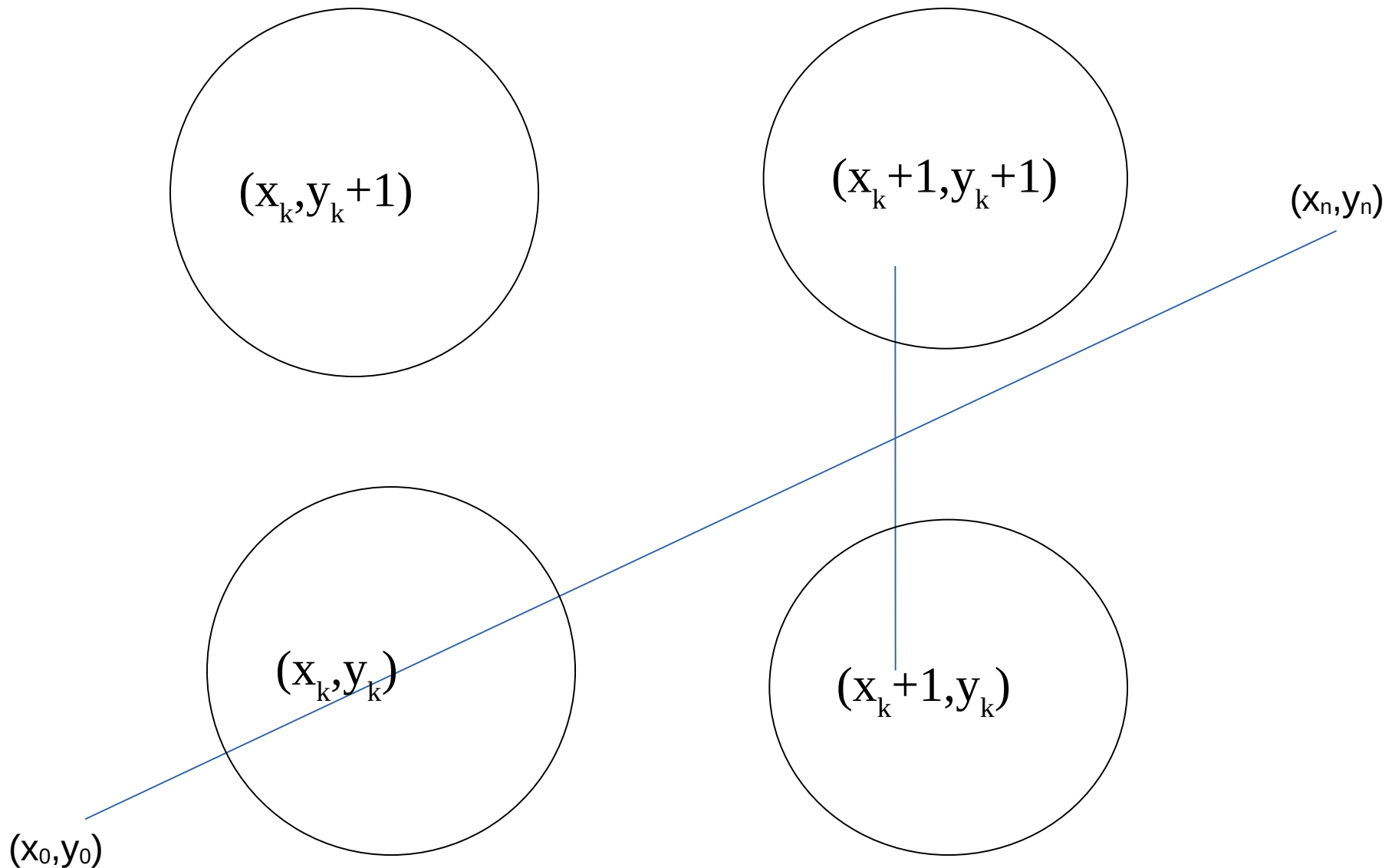
So, next point to be plotted can be either

$SE(x_k + 1, y_k)$ or $S(x_k + 1, y_k + 1)$

Bresenham chooses the closest point from the line as the next point.

Also remember that the line can pass through either one of these points or from middle of these points.

Bresenham Algorithm



Bresenham Algorithm

Let the line pass through $M(x_k + 1, y)$ between SE and S

The distance between $S(x_k + 1, y_k + 1)$ and $M(x_k + 1, y)$ is

$$d1 = \sqrt{((x_k + 1) - (x_k + 1))^2 + (y_k + 1 - y)^2}$$

$$\text{or, } d1 = y_k + 1 - y$$

since line passes through $M(x_k + 1, y)$

$$\text{So, } y = m(x_k + 1) + c$$

$$\text{Now, } d1 = y_k + 1 - m(x_k + 1) - c$$

The distance between $M(x_k + 1, y)$ and $S(x_k + 1, y_k)$ is

$$d2 = \sqrt{((x_k + 1) - (x_k + 1))^2 + (y - y_k)^2}$$

$$\text{or, } d2 = y - y_k$$

$$\text{or, } d2 = m(x_k + 1) + c - y_k$$

Bresenham Algorithm

Lets us evaluate $d2 - d1$

$$d2 - d1 = m(x_k + 1) + c - y_k - y_k - 1 + m(x_k + 1) + c$$

$$\text{or, } d2 - d1 = 2m(x_k + 1) + 2c - 2y_k - 1$$

$$\text{or, } d2 - d1 = 2\frac{dy}{dx}(x_k + 1) + 2c - 2y_k - 1$$

Let us eliminate complex division.

$$(d2 - d1)dx = 2(x_k + 1)dy + 2c.dx - 2y_k.dx - dx$$

$$\text{or, } (d2 - d1).dx = 2x_k.dy + 2dy + 2c.dx - 2y_k.dx - dx$$

Here, $2dy + 2c.dx - dx$ is constant.

Does not contribute in decision making.

So, we can discard them.

Bresenham Algorithm

Now, $(d2 - d1).dx = 2x_k.dy - 2y_k.dx$

Let $p_k = (d2 - d1).dx = 2x_k.dy - 2y_k.dx$
be current decision parameter.

Next decision parameter is

$$p_{k+1} = 2x_{k+1}.dy - 2y_{k+1}.dx$$

Notice that $dx < 0$ is not true, since $dx = |x_n - x_0|$.
So, if $p_k < 0$, it means $d2 < d1$.

This makes $SE(x_k + 1, y_k)$ closer to line.
And bresenham chooses $SE(x_k + 1, y_k)$
as next point on line.

So, $y_{k+1} = y_k$ and $x_{k+1} = x_k + 1$
Now, $p_{k+1} = 2(x_k + 1).dy - 2y_k.dx$
or, $p_{k+1} = 2x_k.dy + 2dy - 2y_k.dx$
or, $p_{k+1} = p_k + 2dy$

Bresenham Algorithm

if $p_k \geq 0$ means $d2 > d1$.

So, $y_{k+1} = y_k + 1$ and $x_{k+1} = x_k + 1$

Now, $p_{k+1} = 2(x_k + 1).dy - 2(y_k + 1).dx$

or, $p_{k+1} = 2x_k.dy + 2dy - 2y_k.dx - 2dx$

or, $p_{k+1} = p_k + 2dy - 2dx$

We have, $p_k = (d2 - d1).dx = 2(x_k + 1).dy + 2c.dx - 2y_k.dx - dx$

For initial decision parameter, put $k = 0$

So, $p_0 = (d2 - d1).dx = 2(x_0 + 1).dy + 2c.dx - 2y_0.dx - dx$

Also, we have, $c = y_0 - \frac{dy}{dx}x_0$

Substituting c we have,

$$p_0 = 2(x_0 + 1).dy + 2(y_0 - \frac{dy}{dx}x_0).dx - 2y_0.dx - dx$$

$$p_0 = 2(x_0 + 1).dy + 2dx \frac{y_0 dx - x_0 dy}{dx} - 2y_0.dx - dx$$

$$p_0 = 2x_0.dy + 2dy + 2y_0.dx - 2x_0.dy - 2y_0.dx - dx$$

$$p_0 = 2dy - dx$$

Bresenham

- If $dy > dx$ i.e. $|m| > 1$ then, role of x and y is exchanged.
- y -coordinate is either incremented or decremented.
- Then x is either kept same or incremented or decremented according to x_0 and x_n .

Bresenham Pseudocode

- Bresenham(x_1, y_1, x_2, y_2)
- 1. $dx = |x_2 - x_1|$, $dy = |y_2 - y_1|$
- 2. $xinc = 1$, $yinc = 1$
- 3. IF $x_1 > x_2$, $xinc = -1$
- 4. IF $y_1 > y_2$, $yinc = -1$
- 5. IF $dx > dy$, $step = dx$
- 6. ELSE $step = dy$
- 7. $x = x_1$, $y = y_1$ and $i = 0$

Bresenham Pseudocode

- 8. drawPixel(x,y)
- 9. IF $dx > dy$
- 10. $p = 2dy - dx$
- 11. WHILE $i < \text{step}$
- 12. $x = x + xinc$
- 13. IF $p < 0$, $p = p + 2dy$
- 14. ELSE $y = y + yinc$, $p = p + 2dy - 2dx$
- 15. drawPixel(x,y)

Bresenham Pseudocode

- 16. $i += 1$
- 17. END WHILE
- 18. ELSE
- 19. $p = 2dx - dy$
- 20. WHILE $i < \text{step}$
- 21. $y = y + y_{\text{inc}}$
- 22. IF $p < 0$, $p = p + 2dx$
- 23. ELSE $x = x + x_{\text{inc}}$, $p = p + 2dx - 2dy$
- 24. drawPixel(x, y)

Bresenham Pseudocode

- 25. $i += 1$
- 26. END WHILE
- 27. END IF

Bresenham

- If $|dy| > |dx|$ i.e. $|m| > 1$ then, role of x and y is exchanged.
- y -coordinate is either incremented or decremented.
- Then x is either kept same or incremented or decremented according to x_0 and x_n .

Bresenham Example

- $(x_1, y_1) = (2, 3)$ and $(x_2, y_2) = (7, 5)$
- $dx = |x_2 - x_1| = |7 - 2| = 5$
- $dy = |y_2 - y_1| = |5 - 3| = 2$
- Since, $dx > dy$,
- $step = dx = 5$
- Since, $x_1 < x_2$, $xinc = 1$
- Since, $y_1 < y_2$, $yinc = 1$
- $p = 2dy - dx = 2 * 2 - 5 = -1$
- $x = x_1$, $y = y_1$, $i = 0$

i	$x += xinc$	If $p < 0$, $y = y$ Else $y += yinc$	if $p < 0$, $p = p + 2dy$, Else, $p = p + 2dy - 2dx$	(x,y)
	2	3	-1	(2,3)
0	$2 + 1 = 3$	3	$-1 + 2x_2 = 3$	(3,3)
1	$3 + 1 = 4$	$3 + 1 = 4$	$3 + 2x_2 - 2x_5 = -3$	(4,4)
2	5	4	1	(5,4)
3	6	5	-5	(6,5)
4	7	5	-1	(7,5)

Bresenham Example

- $(x_1, y_1) = (2, 3)$ and $(x_2, y_2) = (1, 9)$
- $dx = |x_2 - x_1| = |1 - 2| = 1$
- $dy = |y_2 - y_1| = |9 - 3| = 6$
- Since, $dy > dx$,
- $step = dy = 6$
- Since, $x_1 > x_2$, $x_{inc} = -1$
- Since, $y_1 < y_2$, $y_{inc} = 1$
- $p = 2dx - dy = 2 * 1 - 6 = -4$
- $x = x_1$, $y = y_1$, $i = 0$

i	If $p < 0$, $x = x$ Else $x += x_{inc}$	$y += y_{inc}$	if $p < 0$, $p = p + 2dx$, Else, $p = p + 2dx - 2dy$	(x,y)
	2	3	-4	(2,3)
0	2	4	-2	(2,4)
1	2	5	0	(2,5)
2	1	6	-10	(1,6)
3	1	7	-8	(1,7)
4	1	8	-6	(1,8)
5	1	9	-4	(1,9)

Bresenham Example

- $(x_1, y_1) = (35, 40)$ and $(x_2, y_2) = (43, 45)$
- $dx = |x_2 - x_1| = |43 - 35| = 8$
- $dy = |y_2 - y_1| = |45 - 40| = 5$
- Since, $dx > dy$,
- $step = dx = 8$
- Since, $x_1 < x_2$, $xinc = 1$
- Since, $y_1 < y_2$, $yinc = 1$
- $p = 2dy - dx = 2 * 5 - 8 = 2$
- $x = x_1$, $y = y_1$, $i = 0$

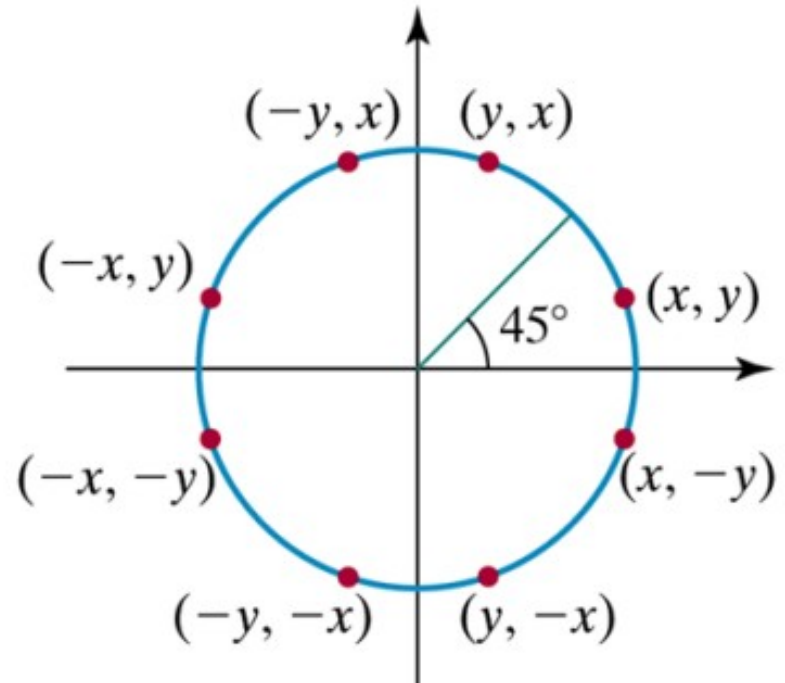
i	$x += xinc$	if $p < 0$, $y = y$ Else, $y += yinc$	if $p < 0$, $p = p + 2dy$, Else, $p = p + 2dy - 2dx$	(x,y)
	35	40	2	(35,40)
0	36	41	-4	(36,41)
1	37	41	6	(37,41)
2	38	42	0	(38,42)
3	39	43	-6	(39,43)
4	40	43	4	(40,43)
5	41	44	-2	(41,44)
6	42	44	8	(42,44)
7	43	45	2	(43,45)

Advantages of Bresenham

- All the computations are integer
- All floating points are removed
- No round function.
- Huge improvement in terms of speed of computation which is desirable
- Because screen is to be refreshed at very high speed to avoid flickr.

Rendering Circle

- As we know circle as a center (x_c, y_c) and radius r .
- The distance between any point (x, y) on circle and center (x_c, y_c) is always r .
- Each point (x, y) on the circle has 7 symmetric points as shown below



Rendering Circle

- So, to render a circle, we need to calculate only points on the first octant.
- Other all points can be calculated using symmetric method.
- The circle can be have any center and any radius.
- First we will learn how to render the circle center at $(0,0)$ and radius r .
- Other circle center at (x_c, y_c) can be rendered by translating the center to origin, calculate octant and the translate back to original center.

Rendering Circle

The Equation of the circle with center $(0, 0)$ and radius be r is
$$x^2 + y^2 - r^2 = 0$$

Let $f(x, y) = x^2 + y^2 - r^2$

if $f(x, y) > 0$, *then* (x, y) lies outside of circle.

if $f(x, y) = 0$, *then* (x, y) lies on the circle.

if $f(x, y) < 0$, *then* (x, y) lies inside the circle.

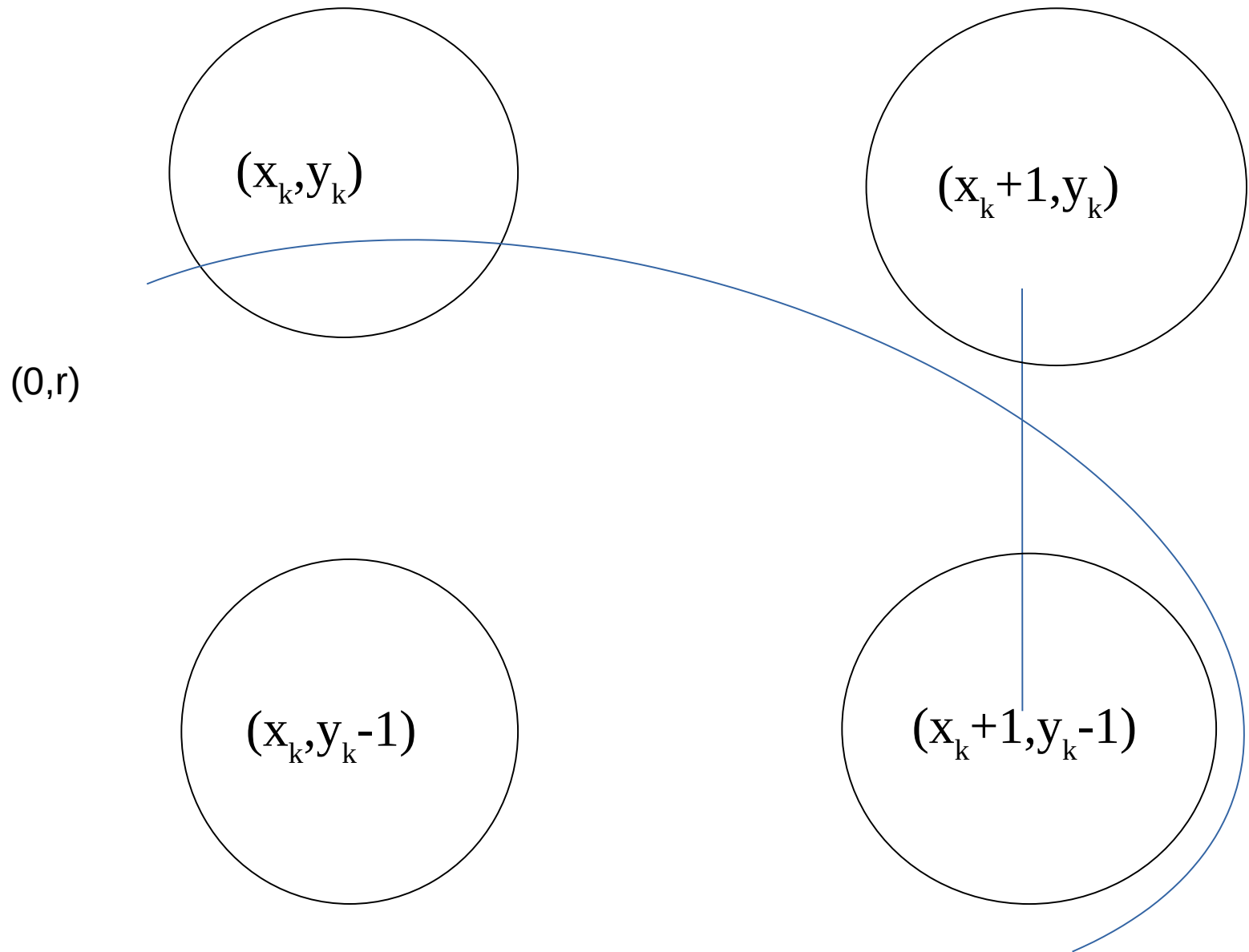
Mid point Theorem

- We can calculate the point in the first octant of circle using midpoint theorem.
- Midpoint theorem plots the points of the circle starting from $(0,r)$.
- It samples the closest point to the circle from next possible two points.
- The closeness of a point to the circle is decided using mid point decision parameter based on mid point of the next possible two points.

Mid point Theorem

- Let us consider a circle of radius r and center at (x_c, y_c) .
- Assume that we have translated the center (x_c, y_c) to origin $(0,0)$
- Also assume that we have started to plot the circle from $(0,r)$ to some point (x_k, y_k) on the circle as shown in the figure below.
- The next possible points are $S(x_k+1, y_k)$ and $SE(x_k+1, y_k-1)$ as shown in the figure
- The midpoint is $M(x_k+1, y_k-1/2)$

Mid Point Theorem



Mid point Theorem

let $p_k = f(x_k + 1, y_k - 1/2) = (x_k + 1)^2 + (y_k - 1/2)^2 - r^2$
be the current decision parameter.

And let $p_{k+1} = (x_{k+1} + 1)^2 + (y_{k+1} - 1/2)^2 - r^2$.
be the next decision parameter.

if $p_k \geq 0$, then M lies outside, so SE is closer to circle.

So, $x_{k+1} = x_k + 1$ and $y_{k+1} = y_k - 1$

$$p_{k+1} = (x_{k+1} + 1)^2 + (y_{k+1} - 1/2)^2 - r^2$$

$$p_{k+1} = (x_k + 1 + 1)^2 + (y_k - 1 - 1/2)^2 - r^2$$

$$= (x_k + 1)^2 + 2(x_k + 1) + 1 + (y_k - 1/2)^2 - 2(y_k - 1/2) + 1 - r^2$$

$$= (x_k + 1)^2 + (y_k - 1/2)^2 - r^2 + 2(x_k + 1) - 2(y_k - 1/2) + 2$$

$$p_{k+1} = p_k + 2(x_k + 1) - 2(y_k - 1/2) + 2$$

$$p_{k+1} = p_k + 2x_{k+1} - 2(y_k - 1 + 1/2) + 2$$

$$p_{k+1} = p_k + 2x_{k+1} - 2y_{k+1} - 1 + 2$$

$$p_{k+1} = p_k + 2x_{k+1} - 2y_{k+1} + 1$$

Mid point Theorem

if $p_k < 0$, then M lies inside, so S is closer to circle.

So, $x_{k+1} = x_k + 1$ and $y_{k+1} = y_k$

$$p_{k+1} = (x_{k+1} + 1)^2 + (y_{k+1} - 1/2)^2 - r^2$$

$$p_{k+1} = (x_k + 1 + 1)^2 + (y_k - 1/2)^2 - r^2$$

$$p_{k+1} = (x_k + 1)^2 + 2(x_k + 1) + 1 + (y_k - 1/2)^2 - r^2$$

$$p_{k+1} = (x_k + 1)^2 + (y_k - 1/2)^2 - r^2 + 2(x_k + 1) + 1$$

$$p_{k+1} = p_k + 2(x_k + 1) + 1$$

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Mid point Theorem

For initial decision parameter, we use initial point $(0, r)$.

$$p_0 = (x_0 + 1)^2 + (y_0 - 1/2)^2 - r^2$$

$$p_0 = (0 + 1)^2 + (r - 1/2)^2 - r^2$$

$$p_0 = 1 + r^2 - r + 1/4 - r^2$$

$$p_0 = 5/4 - r$$

Mid point Theorem

- circleMidPoint(xc,yc,r)
- 1. $p=1-r$
- 2. $x=0, y=r$
- 3. drawCircle(xc,yc,x,y)
- 4. WHILE $x < y$
- 5. $x=x+1$
- 6. IF $p < 0$, $p=p+2x+1$
- 7. ELSE $y=y-1, p=p+2x-2y+1$
- 8. drawCicle(xc,yc,x,y)
- 9. END WHILE

Mid point Theorem

- `drawCircle(xc,yc,x,y)`
- 1. `drawPixel(xc+x,yc+y)`
- 2. `drawPixel(xc+x,yc-y)`
- 3. `drawPixel(xc-x,yc+y)`
- 4. `drawPixel(xc-x,yc-y)`
- 5. `drawPixel(xc+y,yc+x)`
- 6. `drawPixel(xc+y,yc-x)`
- 7. `drawPixel(xc-y,yc+x)`
- 8. `drawPixel(xc-y,yc-x)`

Circle Midpoint Example

- $(x_c, y_c) = (0, 0)$
- $r = 10$
- $p = 1 - r$
- $= 1 - 10 = -9$

$x = x + 1$	if $p < 0$ $y = y$ Else $y = y - 1$	if $p < 0$, $p = p + 2x + 1$ Else $p = p + 2x + 1 - 2y$	$(x + x_c, y + y_c)$
0	10	-9	(0, 10)
1	10	-6	(1, 10)
2	10	-1	(2, 10)
3	10	6	(3, 10)
4	9	-3	(4, 9)
5	9	8	(5, 9)
6	8	5	(6, 8)
7	7	6	(7, 7)

Circle Midpoint Example

- $(x_c, y_c) = (5, 5)$
- $r = 8$
- $p = 1 - r$
- $= 1 - 8 = -7$

$x = x + 1$	if $p < 0$ $y = y$ Else $y = y - 1$	if $p < 0$, $p = p + 2x + 1$ Else $p = p + 2x + 1 - 2y$	$(x + x_c, y + y_c)$
0	8	-7	(5, 13)
1	8	-4	(6, 13)
2	8	1	(7, 13)
3	7	-6	(8, 12)
4	7	3	(9, 12)
5	6	2	(10, 11)